

```

"""
APEX-KV — LSH Band Index
O(1) amortised fuzzy lookup
Replaces the O(N) linear scan in
APEXKVCache.fuzzy_get()

Theory:
    MinHash signature = [h0, h1, h2, h3, h4, h5, h6,
h7] (8 values)
    Split into B bands of R rows each (B*R = 8)
    Two items collide in a band iff ALL R rows match
in that band.
     $P(\text{collision}) \approx 1 - (1 - s^R)^B$  where  $s$  = Jaccard
similarity

    Tuning:
        B=4, R=2:  $P(\text{collision}|s=0.5) \approx 0.94$  — high
recall, more false positives
        B=2, R=4:  $P(\text{collision}|s=0.5) \approx 0.56$  — lower
recall, fewer false positives

    We use B=4, R=2 for maximum recall at our working
similarity range (~0.7).

Lookup: O(B) bucket probes = O(1) amortised
regardless of cache size.
Insert: O(B) bucket inserts = O(1) amortised.
"""

import time, random, math, statistics
from collections import defaultdict

# — LSH Band Index


---


class LSHBandIndex:
    """

```

```

    Replaces linear fuzzy_get scan with O(B) band-
    bucket lookup.
    B=4 bands, R=2 rows each → 8 MinHash values
    total.
    """
    def __init__(self, n_bands=4, n_rows=2):
        assert n_bands * n_rows == 8, "n_bands *
n_rows must equal MinHash length (8)"
        self.B = n_bands
        self.R = n_rows
        # B separate hash tables: band_idx →
        {band_hash → [entry_keys]}
        self.tables = [defaultdict(list) for _ in
range(n_bands)]
        # Master store: lsh_key (int) → value dict
        self.store = {}
        self.hits = 0
        self.misses = 0
        self._probe_count = 0 # total bucket
        probes (for benchmarking)

    def _band_keys(self, minhash_8: list) -> list:
        """Split 8-value MinHash into B band
        keys."""
        keys = []
        for b in range(self.B):
            band_vals = tuple(minhash_8[b*self.R :
b*self.R + self.R])
            keys.append(hash(band_vals)) # Python
            hash of tuple
        return keys

    def insert(self, lsh_key: int, minhash_8: list,
value: dict):
        """O(B) insert."""
        band_keys = self._band_keys(minhash_8)
        for b, bk in enumerate(band_keys):
            if lsh_key not in self.tables[b][bk]:
                self.tables[b][bk].append(lsh_key)
        self.store[lsh_key] = value

```

```

def query(self, minhash_8: list) -> tuple:
    """
    O(B) lookup: probe B buckets, return best
    candidate by Hamming distance.
    Returns (value, hamming_dist) or (None,
    None).
    """
    band_keys = self._band_keys(minhash_8)
    candidates = set()
    for b, bk in enumerate(band_keys):
        self._probe_count += 1
        for ck in self.tables[b].get(bk, []):
            candidates.add(ck)

    if not candidates:
        self.misses += 1
        return None, None

    # Among candidates, pick best by Hamming
    distance on the 4-byte int key
    query_int = _minhash_to_int(minhash_8)
    best, best_d = None, 33
    for ck in candidates:
        d = bin(query_int ^ ck).count('1')
        if d < best_d:
            best_d = d; best = ck

    if best is not None:
        self.hits += 1
        return self.store[best], best_d
    self.misses += 1
    return None, None

def __len__(self): return len(self.store)
def mean_probe_ops(self):
    total = self.hits + self.misses
    return self._probe_count / total if total
else 0.0

```

```

def _minhash_to_int(minhash_8):
    """Collapse 8-value MinHash to 32-bit int for
    Hamming comparison."""
    return int.from_bytes(bytes([x & 0xFF for x in
    minhash_8[:4]]), 'big')

# ——— Corrected APEXKVCache using LSH Band Index
—————

SEEDS =
[0x1fa3,0x2bc7,0x3de1,0x4ef5,0x5f09,0x601d,0x7131,0
x8245]

def fast_simhash(ptrs, bits=32):
    votes = [0]*bits
    for p in ptrs:
        h = 2166136261
        for byte in [(p>>s)&0xFF for s in
(0,8,16)]:
            h ^= byte; h = (h * 16777619) &
0xFFFFFFFF
            h ^= h>>16; h = (h*0x45d9f3b)&0xFFFFFFFF
            h ^= h>>16; h &= 0xFFFFFFFF
            for b in range(bits): votes[b] += 1 if
(h>>b)&1 else -1
        s=0
        for b in range(bits):
            if votes[b]>0: s|=(1<<b)
    return s

def fast_minhash_8(ptrs):
    """Returns full 8-value MinHash list (needed
    for band indexing)."""
    mins = [0xFFFFFFFF]*8
    for p in ptrs:
        for i in range(8):
            h = ((SEEDS[i]*p + SEEDS[(i+3)%8]) ^
(p>>7)) & 0xFFFFFFFF
            if h < mins[i]: mins[i] = h

```

```

        return mins

from collections import OrderedDict

class APEXKVCacheV2:
    """
    APEX-KV cache with O(1) amortised LSH band
    index.
    Drops the O(N) linear scan entirely.
    """
    def __init__(self, cap=512, n_bands=4,
n_rows=2):
        self.cap = cap
        self.exact = OrderedDict()          #
simhash → value (exact tier)
        self.lsh_index = LSHBandIndex(n_bands,
n_rows) # O(1) fuzzy tier
        self.hits_exact = 0
        self.hits_fuzzy = 0
        self.misses = 0
        self.collisions = 0
        self._evict_count = 0

    def _evict_if_full(self):
        if len(self.exact) >= self.cap:
            self.exact.popitem(last=False)
            self._evict_count += 1
        # Note: LSH index is not evicted
    (acceptable: stale candidates
        # are harmless – worst case a fuzzy lookup
returns a stale entry
        # which the caller can validate against the
exact store)

    def get_exact(self, sh_key: int):
        if sh_key in self.exact:
            self.exact.move_to_end(sh_key)
            self.hits_exact += 1
            return self.exact[sh_key]
        return None

```

```

    def get_fuzzy(self, minhash_8: list):
        """O(B) band index probe – replaces O(N)
        scan."""
        return self.lsh_index.query(minhash_8)

    def put(self, sh_key: int, minhash_8: list,
            value: dict):
        if sh_key in self.exact:
            if self.exact[sh_key].get('ptrs') !=
            value.get('ptrs'):
                self.collisions += 1
                self.exact.move_to_end(sh_key)
        else:
            self._evict_if_full()
            self.exact[sh_key] = value
            # Always insert into LSH index (idempotent)
            lsh_int = _minhash_to_int(minhash_8)
            self.lsh_index.insert(lsh_int, minhash_8,
            value)

    def hit_rate(self):
        t = self.hits_exact + self.hits_fuzzy +
        self.misses
        return (self.hits_exact + self.hits_fuzzy)
        / t if t else 0.0

    def memory_bytes(self):
        return sum(len(str(v).encode()) for v in
        self.exact.values())

    def mean_probe_ops(self):
        return self.lsh_index.mean_probe_ops()

# ——— Correctness test
_____

def test_correctness():
    print("—— CORRECTNESS TESTS
    _____")

```

```

idx = LSHBandIndex(n_bands=4, n_rows=2)
random.seed(42)

# Insert 100 items
items = []
for i in range(100):
    mh = [random.randint(0, 0xFFFFFFFF) for _
in range(8)]
    items.append(mh)
    key = _minhash_to_int(mh)
    idx.insert(key, mh, {'id': i})

# Query each item – should find itself (exact
band match)
found = sum(1 for mh in items if idx.query(mh)
[0] is not None)
print(f" Self-recall (should be 100/100):
{found}/100")

# Query with slight perturbation – change last
4 values (different bands)
perturbed_found = 0
for mh in items[:20]:
    q = mh[:4] + [random.randint(0, 0xFFFFFFFF)
for _ in range(4)]
    v, _ = idx.query(q)
    if v is not None: perturbed_found += 1
print(f" Half-perturbed recall (first 4 bands
match): {perturbed_found}/20 (expect high)")

# Query fully random – should mostly miss
random_found = sum(
    1 for _ in range(100)
    if idx.query([random.randint(0, 0xFFFFFFFF)
for _ in range(8)]) [0] is not None
)
print(f" Random query false-positive rate:
{random_found}/100 (expect low)")

print(f" Mean probe ops per query:

```

```

{idx.mean_probe_ops():.1f} (should be ≤ {idx.B})")
    assert found == 100, "Self-recall failed"
    print(" ✓ All correctness checks passed")

# — Latency benchmark: O(N) vs O(1)
_____

def benchmark_scan_vs_index():
    print("\n— LATENCY: O(N) SCAN vs O(1) BAND
INDEX _____")

    from collections import OrderedDict
    def hamming_int(a, b): return
    bin(a^b).count('1')

    # Old O(N) scan
    class OldFuzzy:
        def __init__(self): self.store =
OrderedDict()
        def insert(self, k, v): self.store[k] = v
        def query(self, k, thresh=4):
            best, bd = None, thresh+1
            for ck, cv in self.store.items():
                d = hamming_int(k, ck)
                if d < bd: bd=d; best=cv
            return best, bd

    N_TRIALS = 200
    SIZES = [16, 64, 256, 512, 1024, 2048, 4096]

    print(f" {'Size':>6} {'O(N) scan μs':>14}
{'O(1) band μs':>14} {'Speedup':>8} {'Probe
ops':>10}")
    print(f" {'-'*6} {'-'*14} {'-'*14}
{'-'*8} {'-'*10}")

    speedups = []
    for sz in SIZES:
        random.seed(99)
        keys = [random.randint(0,0xFFFFFFFF) for _

```



```

in range(sz)]
    mhs = [[random.randint(0,0xFFFFFFFF) for _
in range(8)] for _ in range(sz)]

    old = OldFuzzy()
    new = LSHBandIndex(n_bands=4, n_rows=2)
    for k,mh in zip(keys,mhs):
        old.insert(k, {'x':k})
        new.insert(k, mh, {'x':k})

    probe_key = random.randint(0,0xFFFFFFFF)
    probe_mh = [random.randint(0,0xFFFFFFFF)
for _ in range(8)]

    # Time O(N)
    t0 = time.perf_counter()
    for _ in range(N_TRIALS):
old.query(probe_key)
        t_old = (time.perf_counter()-
t0)/N_TRIALS*1e6

    # Time O(1)
    new._probe_count = 0
    t0 = time.perf_counter()
    for _ in range(N_TRIALS):
new.query(probe_mh)
        t_new = (time.perf_counter()-
t0)/N_TRIALS*1e6

    speedup = t_old/t_new if t_new>0 else
float('inf')
    speedups.append(speedup)
    probes = new._probe_count / N_TRIALS

    flag = "✓" if speedup > 5 else ("~" if
speedup > 2 else "x")
    print(f"  {sz:>6}  {t_old:>14.1f}µs
{t_new:>14.1f}µs  {speedup:>7.1f}×
{probes:>10.1f}  {flag}")

```

```

        print(f"\n Mean speedup:
{statistics.mean(speedups):.1f}× "
              f"(max at largest size:
{speedups[-1]:.1f}×)")
        return speedups

# — Full cache benchmark: V1 (O(N)) vs V2 (O(1))
_____

def benchmark_cache_end_to_end():
    print("\n— END-TO-END CACHE BENCHMARK
_____")

    import torch, torch.nn as nn,
    torch.nn.functional as F, math, gc

    torch.manual_seed(42)
    class MHA(nn.Module):
        def __init__(self,d,h):
            super().__init__()
            self.h=h;self.dh=d//h
            self.qkv=nn.Linear(d,3*d,bias=False)
            self.proj=nn.Linear(d,d,bias=False)
            self.last_w=None
        def forward(self,x,mask=None):
            B,T,C=x.shape;H,D=self.h,self.dh
            q,k,v=self.qkv(x).split(C,dim=2)
            q=q.view(B,T,H,D).transpose(1,2)
            k=k.view(B,T,H,D).transpose(1,2)
            v=v.view(B,T,H,D).transpose(1,2)

            sc=torch.matmul(q,k.transpose(-2,-1))/math.sqrt(D)
            if mask is not None:
                sc=sc.masked_fill(mask==0,float('-inf'))

            w=F.softmax(sc,dim=-1);self.last_w=w.detach()
            return
            self.proj(torch.matmul(w,v).transpose(1,2).contiguous().view(B,T,C))

    class Block(nn.Module):

```

```

        def __init__(self,d,h,ff):
            super().__init__()
            self.attn=MHA(d,h)

self.ff=nn.Sequential(nn.Linear(d,ff),nn.GELU(),nn.
Linear(ff,d))

self.ln1=nn.LayerNorm(d);self.ln2=nn.LayerNorm(d)
    def forward(self,x,mask=None):
        x=x+self.attn(self.ln1(x),mask)
        x=x+self.ff(self.ln2(x))
        return x

class GPT(nn.Module):
    def __init__(self):
        super().__init__()

self.tok=nn.Embedding(256,128);self.pos=nn.Embeddin
g(64,128)

self.blocks=nn.ModuleList([Block(128,4,512) for _
in range(6)])

self.ln=nn.LayerNorm(128);self.head=nn.Linear(128,2
56,bias=False)
    def forward(self,idx):
        B,T=idx.shape

x=self.tok(idx)+self.pos(torch.arange(T))

mask=torch.tril(torch.ones(T,T)).unsqueeze(0).unsqu
eeze(0)
        for b in self.blocks: x=b(x,mask)
        return self.head(self.ln(x))
    def weights(self,idx):
        self.eval()
        with torch.no_grad(): self.forward(idx)
        return [b.attn.last_w for b in
self.blocks]

```

```

model=GPT(); model.eval()

PROMPTS=[
    "The quick brown fox jumps over the lazy dog",
    "SELECT id FROM users WHERE age greater than eighteen",
    "function parseTokens return input split space map trim",
    "The transformer architecture uses attention mechanisms",
    "Entropy measures uncertainty in a probability distribution",
    "Summarize the key points in three concise bullet points",
    "The key value cache stores attention patterns for retrieval",
    "LRU eviction removes least recently used entries from cache",
    "APEX-KV extracts signatures from pointer fields routing",
    "Neural networks learn hierarchical data representations",
    "Compare the two approaches and recommend the better solution",
    "Cache invalidation strategies distributed memory storage",
]

def encode(t,ml=64):
    ids=list(t.encode('utf-8'))[:ml]; return
ids+[0]*(ml-len(ids))

tokens=
[torch.tensor([encode(p)],dtype=torch.long) for p
in PROMPTS]

# Old O(N) cache (from previous benchmark)
from collections import OrderedDict
def hamming_int(a,b): return

```

```

bin(a^b).count('1')

class OldCache:
    def __init__(self, cap=512):

self.cap=cap; self.exact=OrderedDict(); self.lsh=OrderedDict()

self.hits_e=self.hits_f=self.misses=0; self._scan=0
    def get_exact(self, k):
        if k in self.exact:

self.exact.move_to_end(k); self.hits_e+=1; return
self.exact[k]
        return None
    def get_fuzzy(self, k, thresh=4):

best, bd=None, thresh+1; self._scan+=len(self.lsh)
        for ck, cv in self.lsh.items():
            d=hamming_int(k, ck)
            if d<bd: bd=d; best=cv
            if best: self.hits_f+=1; return best, bd
        return None, None
    def put(self, sh, lk, v):
        if len(self.exact)>=self.cap:
self.exact.popitem(last=False)
            self.exact[sh]=v
            if len(self.lsh)>=self.cap:
self.lsh.popitem(last=False)
                self.lsh[lk]=v
    def hit_rate(self):
        t=self.hits_e+self.hits_f+self.misses
        return (self.hits_e+self.hits_f)/t if t
else 0.0

def run_old(toks_list):
    cache=OldCache(512); lats=[]
    for toks in toks_list:
        t0=time.perf_counter()
        weights=model.weights(toks)

```

```

        seq=int((toks[0]!=0).sum())
        hits=misses=0
        for li,lw in enumerate(weights):
            for hi in range(lw.shape[1]):
                for qp in range(min(seq,12)):
                    w=lw[0,hi,qp]

        ptrs=torch.topk(w,min(6,seq)).indices.tolist()
        sh=fast_simhash(ptrs)

        lk=int.from_bytes(bytes([x&0xFF for x in
        fast_minhash_8(ptrs)[:4]]), 'big')
        hit=cache.get_exact(sh)
        if not hit:
            hit,_=cache.get_fuzzy(lk)
            if hit: hits+=1
            else: misses+=1;
        cache.put(sh,lk,{'ptrs':ptrs})
        lats.append(time.perf_counter()-t0)
        return lats, cache

    def run_new(toks_list):

        cache=APEXKVCacheV2(512,n_bands=4,n_rows=2); lats=
        []

        for toks in toks_list:
            t0=time.perf_counter()
            weights=model.weights(toks)
            seq=int((toks[0]!=0).sum())
            hits=misses=0
            for li,lw in enumerate(weights):
                for hi in range(lw.shape[1]):
                    for qp in range(min(seq,12)):
                        w=lw[0,hi,qp]

        ptrs=torch.topk(w,min(6,seq)).indices.tolist()
        sh=fast_simhash(ptrs)
        mh8=fast_minhash_8(ptrs)
        hit=cache.get_exact(sh)
        if not hit:

```

```

hit,_=cache.get_fuzzy(mh8)
        if hit: hits+=1
        else:
            misses+=1
            cache.put(sh,mh8,
{'ptrs':ptrs})
        lats.append(time.perf_counter()-t0)
    return lats, cache

N=10
print(f"  Running {N} trials × {len(PROMPTS)}
prompts each...")

old_lats=[]; new_lats=[]; old_hr=[]; new_hr=[]
for _ in range(N):
    gc.collect()
    l,c=run_old(tokens);
old_lats.append(sum(l));
old_hr.append(c.hit_rate())
    l,c=run_new(tokens);
new_lats.append(sum(l));
new_hr.append(c.hit_rate())

from scipy import stats as sp

t,p=sp.ttest_ind(new_lats,old_lats,equal_var=False)
total_tok=sum(int((t[0]!=0).sum()) for t in
tokens)

def tps(lats): return [total_tok/l for l in
lats]
old_tps=tps(old_lats); new_tps=tps(new_lats)

print(f"\n  {'Metric':<22} {'Old O(N)':>14}
{'New O(1)':>14} {'Δ':>8} {'p-value':>10}")
print(f"  {'-'*22} {'-'*14} {'-'*14} {'-'*8}
{'-'*10}")

def pr(name,a,b,unit="",higher_better=False):
    ma,mb=statistics.mean(a),statistics.mean(b)

```

```

sa,sb=statistics.stdev(a),statistics.stdev(b)
delta=(mb-ma)/ma*100
t2,p2=sp.ttest_ind(b,a,equal_var=False)
better="✓" if (higher_better and mb>ma) or
(not higher_better and mb<ma) else "✗"
print(f" {name:<22} {ma:>10.1f}±{sa:.1f}
{unit:<3} {mb:>10.1f}±{sb:.1f}{unit:<3} "
      f"{delta:>+7.1f}% p={p2:.4f}
{better}")

pr("Latency (ms)",
    [l*1000 for l in old_lats],[l*1000 for l in
new_lats],"ms",False)
pr("Tokens/sec",old_tps,new_tps," ",True)
pr("Hit rate (%)",
    [h*100 for h in old_hr],[h*100 for h in
new_hr],"%",True)

# Probe ops
_,c_new=run_new(tokens)
_,c_old=run_old(tokens)
print(f"\n O(N) scan ops/lookup (old):
{c_old._scan/(c_old.hits_f+c_old.misses+1):.1f}")
print(f" Band probe ops/lookup (new):
{c_new.mean_probe_ops():.1f} (fixed =
{c_new.lsh_index.B})")

return old_lats, new_lats

if __name__ == "__main__":
    print("="*60)
    print("LSH BAND INDEX – O(1) FUZZY LOOKUP")
    print("Eliminating the O(N) bottleneck")
    print("="*60)

    test_correctness()
    benchmark_scan_vs_index()
    benchmark_cache_end_to_end()

```



```
print("\n" + "="*60)
print("DONE")
print("="*60)
```